

Streaming Live RAG: Documentation Pack

Track: Theme 4, Streaming Live RAG (real-time incremental retrieval, multi-intent decomposition, state-preserving answer refinement)

Contents

- Part A: Detailed problem description (what the track really asks)
- Part B: Product Requirements Document (PRD)
- Part C: System Requirements / Specification Document (SRD)
- Part D: How to build the proper docs (templates, page budgets, review checklist)
- Appendix: Traceability matrix, repo layout, 2-week plan

Convention used below. Anything taken directly from the track guide is stated as fact. Anything marked [Proposed] is our own design choice or target, because the guide doesn't specify it. Anything marked [Clarify] is an open question to confirm with the organizers.

PART A: Detailed Problem Description

A1. The problem in one paragraph

Normal RAG is turn-based: the user finishes, the system retrieves, the system answers. In voice and live-support settings, that means dead air. This track asks you to build an event-driven engine that consumes a transcript as it arrives (timestamped chunks such as 0.0 s, 0.8 s, 1.6 s, then "utterance end" at 2.1 s), starts retrieving before the user finishes, splits one rambling utterance into several sub-questions, updates its answer in place when the user adds a late detail, and never says anything that isn't backed by the supplied corpus, with full telemetry of everything it did.

A2. Why standard RAG fails here

Failure	What happens in a batch RAG	What the track wants instead
High latency	Waits for the whole utterance, then retrieves, so the user waits several seconds	Retrieval begins mid-utterance once intent is stable
Compound requests	One blended query such as "venue capacity cancellation catering" retrieves mush	Decompose into orthogonal sub-queries, retrieve in parallel
Late constraints	"Actually it was international" restarts the pipeline and discards context	Mutate only the affected claims; keep the rest and the citations

A3. The four required capabilities, explained

1. Listens incrementally (Retrieval Controller). For every incoming chunk, the controller makes one of three decisions:

- Wait: the intent is incomplete or unstable (the 0.0 s chunk "I need to plan a customer workshop in..." has no location yet).
- Retrieve: the intent is stable enough to search (at 0.8 s, "Pune for 30 people" gives a location and a capacity).
- No-Retrieval / Suppress: the request doesn't need the corpus at all (e.g. "repeat that in two bullets").

The hard part is the trade-off: retrieve too early and you burn compute on half-thoughts (the guide calls this *thrashing*); retrieve too late and you gain nothing over batch RAG.

2. Decomposes multi-intent queries. "...the cancellation policy and the catering options" contains two questions. Together with the earlier capacity question, that makes three sub-queries, dispatched in parallel. The trap is over-fragmenting: a simple question must not be split into near-duplicate queries (Pitfall #5).

3. Refines rather than restarts (Session-Only Refinement). The answer is a *versioned object*. When the user adds "the trip was international and the booking was made after travel", you do not rerun everything. You:

1. detect that this modifies an existing topic,
2. query only for the delta (international travel rules, post-travel booking exceptions),
3. rewrite only the affected claims,
4. keep the earlier citations, add new ones, and bump Answer Version 1 → 2.

4. Guarantees corpus grounding. Every factual claim must carry a citation that exists in the corpus and actually supports the claim. If evidence is missing, the system must say so (an explicit uncertainty indicator) or ask a targeted clarification. Never invent a citation such as .

A4. The pipeline, stage by stage

Stream chunks → [1] Retrieval Controller → [2] Multi-Intent Decomposer
→ [3] Corpus Retrieval & Fusion → [4] Session-Aware Synthesis
→ Streamed answer + grounded citations + telemetry

Stage	Input	Output	Core challenge (from the guide)
1. Retrieval Controller	Transcript chunks + session state	Wait / Retrieve / No-Retrieval	Early latency gains vs premature noisy searches
2. Multi-Intent Decomposer	Utterance so far	List of search-ready sub-queries	Orthogonal questions without losing context or over-fragmenting
3. Retrieval & Fusion	Sub-queries	Ranked, deduplicated chunks per sub-query	Merge sources without diluting context or introducing contradictions
4. Session-Aware Synthesis	Evidence + prior answer version	Streamed answer, citations, uncertainty flags	Version tracking; mutate only affected claims

Cross-cutting: Telemetry	Everything	Structured logs	Clean telemetry under sub-second streaming
-----------------------------	------------	-----------------	---

A5. Hard rules and what they mean for design

Rule (guide §3)	Design implication
Corpus isolation. No web, no third-party KBs, no parametric memory for facts.	An LLM may only <i>rephrase and combine retrieved text</i> . Prompt it with retrieved chunks only and verify outputs against them.
No hardcoding / precomputation. The replay benchmark is held-out and private.	No canned answers, no query-specific rules. Only generic indexing of the supplied corpus at startup is safe. [Clarify] whether indexing the corpus at build time is allowed.
Rigorous grounding. Every claim cites ; otherwise uncertainty or clarification.	Claim-level data model, plus a verifier that runs <i>before</i> anything is emitted.
Session-bound state. No cross-session profiling; memory is ephemeral.	In-memory store keyed by , wiped at session end; nothing persisted between runs.
Architectural parsimony. Extra agents and orchestration are judged on cost-to-performance.	Prefer a single async pipeline. Every added component needs a measured latency/quality justification (ablations!).

A6. What the three worked examples teach

- Example 1 (incremental multi-intent): Timeline at 0.0 s Wait → 0.8 s provisional retrieve (, logged as) → 1.6 s decompose into 3 sub-queries and search in parallel → 2.1 s utterance end, synthesize one unified answer with citations. The output includes an field for anything not verifiable (e.g. catering policy for Venue A).
- Example 2 (late detail): Version 1 answers the base policy; the late detail triggers targeted delta queries; Version 2 keeps prior citations, adds new ones, and states the exception clearly.
- Example 3 (suppression): "Repeat your last answer in two bullets" → , ; zero vector searches, prior citations retained, nothing new fabricated.

A7. The six evaluation gates, decoded

Gate	Threshold	What judges will likely check	How we hit it
G1 Reproducibility	Pass/Fail	Fresh machine, one command, replay suite runs unattended	Docker Compose; pinned lockfiles; no manual steps; no online API needed at runtime (or clear key config)

G2 Early Retrieval	≥ 80% of eligible queries	timestamp < utterance-end timestamp, with a low false-trigger rate on no-retrieval cases	Feature-based controller; measure the false-trigger rate ourselves
G3 Multi-Intent	≥ 70% of compound utterances	≥ 2 distinct sub-intents identified and isolated	Decomposer with a golden set; guard against both under- and over-splitting
G4 Grounding	≥ 85% citation support; zero fabricated IDs	Sampled claims supported by cited chunks	Claim-level verifier; drop or convert unsupported claims to uncertainty
G5 Refinement	Verified state continuity	Late constraints update without clearing session or re-running full-corpus search	Claim-level answer versioning; delta retrieval only
G6 Telemetry	100% trace coverage	Timestamps, triggers, citations, version lineage, token cost for every request	Central event emitter; schema validated in tests

A8. Pitfalls → mitigations

Pitfall	Mitigation
Eager retrieval on noise	Stability check + per-utterance cap on provisional retrievals + reuse of cached results
Context loss on late constraints	Session store holds evidence and claims; delta-only queries
Citation hallucination	Whitelist of retrievable IDs; verifier rejects any ID not in the retrieved set
Ignoring presentation-only turns	Suppression classifier runs <i>first</i> in the controller
Over-fragmented sub-queries	Merge near-duplicates (similarity threshold); cap sub-query count

A9. Open questions to confirm with organizers

[Clarify]

1. Corpus format and size (files, scheme, how sections are delimited). Is the corpus given in advance or only at evaluation time?
2. Input format of the held-out replay: JSONL of ? Real-time pacing or accelerated replay?
3. Output contract: is the JSON in Example 1 the exact schema the harness reads? Where are telemetry logs read from (file, stdout, endpoint)?

4. LLMs: are external hosted LLM APIs allowed? Local models only? Any compute or cost cap? (This affects G1 heavily.)
 5. Latency numbers: the guide gives none. Is there a target for time-to-first-token after utterance end?
 6. G2 definition: what makes a query "eligible", and what false-trigger rate counts as "low"?
 7. G4 method: human-judged or automated entailment? Granularity (sentence or claim)?
 8. Baseline: is the "baseline pipeline" for the benchmarking report provided by organizers, or do we build one?
 9. Clarification requests: does asking a clarifying question count as an acceptable alternative to an uncertainty flag under G4?
-

PART B: Product Requirements Document (PRD)

B1. Overview

Field	Value
Product	Streaming Live RAG Engine
Purpose	Deliver grounded, low-latency answers to live speech/typing by retrieving incrementally, decomposing compound requests, and refining answers in place
Primary consumer	Hackathon evaluation harness (automated replay), then judges reviewing the demo and docs
Secondary consumer (product vision)	Voice assistants and real-time support agents
Version	0.1 (draft)

B2. Goals and non-goals

Goals

1. Start retrieval before the utterance ends on $\geq 80\%$ of eligible queries, without thrashing on no-retrieval turns.
2. Correctly split compound utterances ($\geq 70\%$) into parallel sub-queries.
3. Produce answers where $\geq 85\%$ of factual claims are supported by valid citations, with zero fabricated IDs.
4. Refine answers in place when late constraints arrive, without clearing state or re-running full-corpus search.
5. Emit complete telemetry for every request (100% trace coverage).
6. Run from a single command on a clean machine.

Non-goals

- Speech-to-text (we consume *transcript chunks*; we don't build ASR). [Clarify]

- Cross-session memory, user profiling, personalization.
- Web search or any knowledge outside the supplied corpus.
- A polished end-user UI (a minimal demo UI or CLI is enough, mainly to show telemetry).
- Multi-agent orchestration beyond what ablations justify.

B3. Users and scenarios

Persona	Need	Example
Voice-assistant end user	Fast, correct answers while still talking; can change their mind mid-conversation	"Plan a workshop in Pune for 30... and the cancellation policy and catering?"
Support agent (assisted)	Grounded snippets with sources while a customer is speaking	Live policy lookup during a call
Evaluation harness	Deterministic replay, machine-readable outputs and logs	Automated gate scoring
Judge / reviewer	Understand design, trade-offs, evidence	Reads brief, report, watches demo

B4. User stories

ID	Story	Priority
US-1	As a speaker, I want the system to start searching once my intent is clear, so I don't wait after I finish.	Must
US-2	As a speaker, I want each part of my compound request answered, so I don't have to repeat myself.	Must
US-3	As a speaker, I want my earlier answer updated when I add a detail, so the conversation flows naturally.	Must
US-4	As a reader, I want every fact linked to a source, and gaps flagged, so I can trust the answer.	Must
US-5	As a speaker, I want formatting requests (shorten, bullets, translate) handled without a new search.	Must
US-6	As an evaluator, I want structured logs of all decisions, so I can score the run.	Must
US-7	As an evaluator, I want to run everything with one command.	Must
US-8	As a developer, I want to compare controller and retrieval variants (ablations).	Should
US-9	As a demo viewer, I want to see the timeline, version history, and citations live.	Should

US-10	As a speaker, I want a clarifying question when evidence is ambiguous.	Could
-------	--	-------

B5. Functional requirements

Retrieval Controller

ID	Requirement	Priority
FR-1.1	Process timestamped transcript chunks in arrival order, emitting a decision (, ,) per chunk with a reason code.	Must
FR-1.2	Trigger a provisional retrieval when the intent is stable (topic plus at least one concrete constraint).	Must
FR-1.3	Suppress retrieval for presentation-only requests (repeat, shorten, reformat, translate, summarize prior output) and log .	Must
FR-1.4	Limit provisional retrievals per utterance and reuse results if the final query is equivalent.	Should
FR-1.5	Reconcile provisional results with the final utterance at utterance end (keep, extend, or discard).	Must

Multi-Intent Decomposition

ID	Requirement	Priority
FR-2.1	Extract independent sub-queries from a compound utterance while preserving shared context (e.g. "Pune", "30 people").	Must
FR-2.2	Dispatch sub-query retrievals concurrently.	Must
FR-2.3	Merge near-duplicate sub-queries; never fragment a single-intent utterance.	Must
FR-2.4	Track each sub-intent as an object (, text, status) for coverage checks.	Must

Retrieval and Fusion

ID	Requirement	Priority
FR-3.1	Retrieve only from the supplied corpus using hybrid (sparse + dense) scoring.	Must
FR-3.2	Fuse ranked lists (e.g. RRF), deduplicate overlapping chunks, and rerank.	Must

FR-3.3	Preserve and provenance on every chunk.	Must
--------	---	------

Synthesis, Refinement, Grounding

ID	Requirement	Priority
FR-4.1	Generate answers as claims, each with ≥ 1 citation .	Must
FR-4.2	Stream the answer incrementally and support versioning ().	Must
FR-4.3	On a late constraint, identify affected claims, run delta retrieval only, mutate those claims, retain the rest, increment the version.	Must
FR-4.4	Verify each claim against its cited chunk before emitting; drop or convert failures to an uncertainty statement.	Must
FR-4.5	Every sub-intent ends with either a supported claim or an explicit uncertainty entry.	Must
FR-4.6	Session memory is ephemeral, scoped to , and cleared at session end.	Must
FR-4.7	Optionally emit a clarifying question when evidence is insufficient or ambiguous.	Could

Observability and Packaging

ID	Requirement	Priority
FR-5.1	Emit structured telemetry events for all decisions (see SRD §3.3).	Must
FR-5.2	Record per-request latency breakdown and token/cost estimates.	Must
FR-5.3	Provide a replay runner to execute test streams and produce metrics.	Must
FR-5.4	Launch with (or a single CLI).	Must
FR-5.5	Provide a minimal live view (CLI or web) showing timeline, versions, citations.	Should

B6. Success metrics

Metric	Target	Source
--------	--------	--------

Early-retrieval rate	$\geq 80\%$ of eligible queries	G2
False-trigger rate on no-retrieval cases	[Proposed] $\leq 10\%$ (confirm)	G2
Multi-intent identification	$\geq 70\%$ of compound utterances	G3
Citation support	$\geq 85\%$ of sampled claims	G4
Fabricated citation IDs	0	G4
State continuity on refinements	100% of refinements avoid session reset and full re-search	G5
Trace coverage	100%	G6
Clean-machine one-command replay	Pass	G1
[Proposed] Time from utterance end to first answer token	≤ 1.0 s at p50 (internal target)	Team

B7. Scope: MVP vs stretch

MVP (must be fully working)

- Rule-based controller with stability heuristics and suppression
- LLM- or rule-assisted decomposer
- Hybrid retrieval + RRF + dedupe
- Claim-level synthesis, verifier, uncertainty
- Answer versioning and delta refinement
- Telemetry JSONL, replay runner, Docker packaging

Stretch

- Model-based controller (for the ablation)
- Cross-encoder reranker
- Live dashboard, clarification questions
- Speculative pre-generation of answer drafts

B8. Assumptions, dependencies, risks

Type	Item	Mitigation
Assumption	Input is text chunks with timestamps	Confirm with organizers
Assumption	Corpus fits in memory and can be indexed at startup	Lightweight indexes, persisted only within the container run
Dependency	LLM for synthesis/decomposition	Pluggable interface; local fallback; determinism (temp 0)
Risk	Hidden replay data differs from our tests	Build diverse synthetic test streams; avoid overfitting/hardcoding
Risk	Verifier too strict, dropping good claims	Tune thresholds on our held-out set; report precision/recall

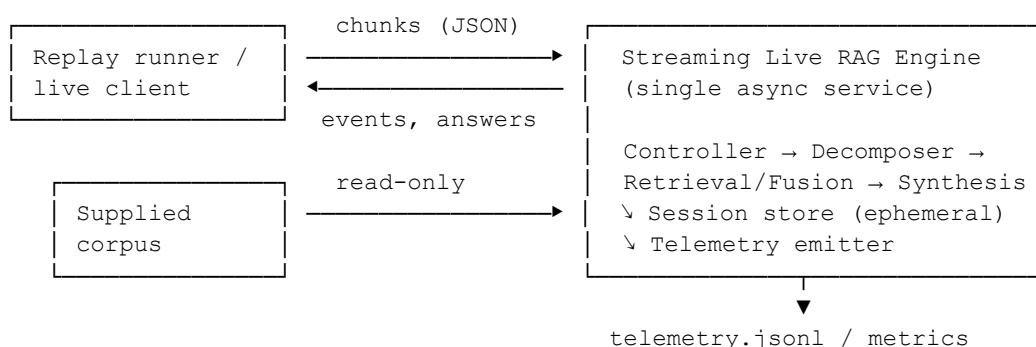
Risk	Latency from LLM calls	Cap calls per utterance; run decomposition and retrieval concurrently
Risk	Over-engineering (parsimony rule)	Ablations justify each component; drop what doesn't pay off

B9. Deliverables (from the guide's §8)

1. Reproducible repository (source, pinned lockfiles, env templates, one-command run)
2. System Architecture Brief (≤ 6 pages)
3. Benchmarking & Evaluation Report (vs baseline; ≥ 3 analyzed edge-case failures; 2 ablations)
4. System Demonstration Video (≤ 5 minutes)
5. Telemetry & Observability Schema

PART C: System Requirements / Specification Document (SRD)

C1. System context



External interfaces: input event stream; output event stream; corpus (read-only); telemetry sink; optional LLM endpoint ([Clarify] whether external APIs are allowed).

C2. Architecture and recommended stack

[Proposed]

Layer	Recommendation	Why
Runtime	Python 3.11,	Concurrency for parallel sub-query retrieval; one process keeps overhead low
API	FastAPI with WebSocket/SSE for live; CLI for replay	Streaming in and out; harness-friendly

Sparse index	BM25 (, SQLite FTS5, or Tantivy)	Exact keyword and ID matches
Dense index	Small local embedding model (e.g. bge-small/e5-small) + FAISS/hnswlib	Offline, reproducible, fast
Fusion	Reciprocal Rank Fusion (k≈60)	Simple, robust, no training
Reranker	Optional cross-encoder (stretch; ablation)	Precision gain vs latency cost
LLM	Pluggable interface (local or hosted), temperature 0, JSON-mode outputs	Decomposition and synthesis; swap for cost/latency tests
State	In-memory dict keyed by	Satisfies ephemeral-memory rule
Schemas	Pydantic models	Validation for I/O and telemetry
Packaging	Dockerfile + , lockfile (/ with hashes)	G1
Testing	, replay harness, metrics scripts	Gates G2–G6

Parsimony stance: start with a *single pipeline, no agent framework*. Add anything else only with an ablation that shows it earns its latency and cost.

C3. Interface specifications

C3.1 Input event [Proposed; align to organizer format]

```
{
  "session_id": "s-001",
  "seq": 3,
  "timestamp_s": 1.6,
  "text": "...the cancellation policy and the catering options.",
  "type": "chunk"           // "chunk" | "utterance_end" | "session_end"
}
```

C3.2 Output event (answer record). Follows the guide's Example 1 and extends it

```
{
  "session_id": "s-001",
  "answer_version": 1,
  "retrieval_events": [
    { "timestamp_s": 0.8, "query": "Pune workshop venue capacity 30", "trigger":
"provisional" },
    { "timestamp_s": 1.6, "query": "cancellation policy workshop venues Pune",
"trigger": "multi_intent" }
  ],
  "sub_queries": ["venue capacity for 30 attendees in Pune", "cancellation terms
and refund policies", "on-site and external catering options"],
  "answer": "For a 30-person workshop in Pune, documented options include ...",
  "claims": [
```

```
    { "claim_id": "c1", "sub_intent_id": "si1", "text": "...", "citations": ["Doc_12
$2"], "status": "supported" }
  ],
  "citations": ["Doc_12 $2", "Doc_31 $4", "Doc_09 $1"],
  "uncertainty": "Catering accommodation policies for Venue A could not be verified
from the retrieved corpus."
}
```

C3.3 Telemetry event schema (one JSON object per line)

```
{
  "ts_wall": "2026-09-21T10:15:02.123Z",    // absolute time
  "ts_stream_s": 0.8,                      // position in the replayed stream
  "session_id": "s-001",
  "request_id": "r-001",
  "event": "retrieval_started",
  "payload": { },
  "latency_ms": null,
  "tokens_in": 0, "tokens_out": 0, "cost_usd": 0.0
}
```

Required event types

Event	Payload highlights
	seq, text, ts_stream_s
	decision (WAIT/RETRIEVE/NO_RETRIEVAL), reason, features (stability score, drift, slot list)
	sub_intents, merged_duplicates_count, latency_ms
/	sub_intent_id, query, trigger (provisional/multi_intent/final/delta), top-k chunk IDs + scores, latency_ms
	method (RRF), input lists, dedupe_removed
/	answer_version, first-token latency, total tokens
	claims_checked, supported, dropped, invalid_citation_count
	from_version, to_version, affected_claim_ids, delta_queries, retained_claim_ids
	reason (e.g.)
	sub_intent_id, cause
	memory_cleared: true

Trace-coverage rule (G6): every request has a complete chain from to , with no gaps, plus version lineage and cost fields.

C4. Component specifications

C4.1 Retrieval Controller

Inputs: chunk, accumulated utterance text, session state. Outputs: decision + reason + features.

Decision order (evaluate top to bottom, first match wins):

1. Suppression check. Presentation-only intent (verbs like repeat, shorten, rephrase, translate, "in N bullets") that refers to a prior answer →
().
2. Refinement check. Utterance refers to an existing topic and adds a constraint ("actually", "also", "the trip was...") → in mode.
3. Stability check. Retrieve provisionally when all hold [Proposed heuristics, tune on our data]:
 - a topic/intent head is present (e.g. "workshop", "reimbursement");
 - ≥ 1 concrete constraint slot filled (location, count, date, product...);
 - the chunk does not end in an unfinished construction (dangling "and I", "in...", "for the");
 - the intent hypothesis has drifted less than a threshold from the last provisional query (embedding or token overlap).
4. Otherwise → .

Budgets: max N provisional retrievals per utterance (start with 2–3); skip re-retrieval if the new query is $\geq \tau$ similar to a cached one. Reconciliation at utterance end: compare final sub-intents with provisional ones; reuse the cache when equivalent, run extra retrievals only for uncovered sub-intents, and drop provisional results that no longer match.

Variants for ablation: rule-based (default) vs. small-model-based classifier.

C4.2 Multi-Intent Decomposer

- Input: utterance text (possibly partial) + session context. Output: list of .
- Method: LLM with strict JSON schema, few-shot examples; deterministic fallback (split on coordinating conjunctions and enumerations, then filter by noun-phrase content).
- Rules: inherit shared context slots into each sub-query (city, headcount); merge sub-queries whose embedding similarity $> \tau$; cap at K sub-queries [Proposed K = 5]; single-intent utterances yield exactly one sub-query.

C4.3 Retrieval and Fusion

1. Query each sub-intent concurrently against sparse and dense indexes (top-k each, e.g. k = 20).
2. Fuse with RRF: .
3. Deduplicate near-identical or overlapping chunks (same or high text overlap).
4. Optional rerank (cross-encoder) → keep top-m per sub-intent (e.g. m = 3–5).
5. Return chunks with , , text, scores.

Corpus ingestion: parse into chunks aligned to sections, so every chunk has a stable

. Build both indexes at startup or at container build [Clarify which is permitted].

C4.4 Session-Aware Synthesis and Refinement

Session state (ephemeral):

```
Session {  
  session_id, answer_version,  
  sub_intents: {id → SubIntent},  
  evidence: {sub_intent_id → [chunk_ids]},
```

```

    claims: [Claim{id, text, sub_intent_id, citations[], status, introduced_in_v,
updated_in_v}],
    answer_history: [version snapshots (diffs)]
}

```

Synthesis: prompt the LLM with *only* retrieved chunks for each sub-intent; require output as claims with citation IDs chosen from the provided set; temperature 0.

Refinement (delta engine):

1. Extract the new constraint(s) from the latest utterance.
2. Map to affected sub-intents/claims (topic match + slot overlap).
3. Run delta retrieval only for the new constraint (e.g.
 ,
).
4. Regenerate or edit only affected claims; add new ones; mark contradicted claims (kept in history).
5. Retain unaffected claims and citations; increment
 ; log

Grounding verifier (runs before emit):

1. ID validity: each citation $\in$ the set of retrieved chunk IDs $\in$ corpus index. Otherwise reject.
2. Support check: claim text is entailed by the cited chunk (embedding/lexical overlap gate, plus an NLI or LLM-judge check for uncertain cases).
3. On failure: rewrite once using the evidence, else drop the claim and record for that sub-intent.
4. Coverage invariant: every sub-intent has ≥ 1 supported claim or an explicit uncertainty entry.

Uncertainty triggers: top retrieval score below threshold, no chunk covering the sub-intent, verifier failure, or conflicting sources.

C4.5 Telemetry Emitter

- Async, non-blocking queue writing JSONL; never awaited on the critical path.
- Every component emits through one helper, so no event is forgotten.
- Token and cost accounting wraps the LLM client.
- A summary script converts logs into a metrics table (G2–G6).

C5. Non-functional requirements

ID	Requirement	Target
NFR-1	Reproducibility	Single command; pinned deps; deterministic seeds; temp 0
NFR-2	Time to first answer token after utterance end	[Proposed] $p_{50} \leq 1.0$ s, $p_{95} \leq 2.0$ s
NFR-3	Controller decision time per chunk	[Proposed] ≤ 50 ms (rule-based)
NFR-4	Retrieval latency per sub-query	[Proposed] ≤ 200 ms on the provided corpus

NFR -5	Isolation	No outbound calls except the (permitted) LLM endpoint; no web; corpus read-only
NFR -6	Privacy	No persistence across sessions; memory cleared on
NFR -7	Robustness	Handles out-of-order/duplicate chunks, empty chunks, very long utterances, corpus-missing topics
NFR -8	Cost	Track tokens per request; report cost-to-quality in the benchmark
NFR -9	Observability	100% trace coverage; schema-validated logs
NFR -10	Maintainability	Type hints, schema validation, ≥ 1 test per requirement group

C6. Verification matrix

Requirement group	Test type	Pass criterion
Controller (FR-1.x)	Replay streams with labeled ideal trigger points	$\geq 80\%$ early trigger; false-trigger rate within target
Decomposition (FR-2.x)	Golden compound utterances	$\geq 70\%$ correct; zero over-fragmentation on single-intent set
Retrieval (FR-3.x)	Golden question→chunk pairs	Recall@k and MRR; hybrid $\geq$ dense-only (ablation)
Grounding (FR-4.1, 4.4, 4.5)	Sampled-claim audit (script + manual)	$\geq 85\%$ supported; 0 invalid IDs
Refinement (FR-4.3)	Two-step scenarios	Version increments; no full-corpus re-search; unaffected claims unchanged
Suppression (FR-1.3)	Presentation-only prompts	0 retrievals; citations preserved
Telemetry (FR-5.x)	Log validator	100% chain completeness
Packaging (FR-5.4)	Clean VM / fresh container	One command reproduces results

C7. Deployment

- builds the image, mounts the corpus read-only, starts the service and (optionally) runs the replay suite, writing and to a mounted folder.
- documents all configuration (model names, thresholds, LLM endpoint).

- README gives a 5-minute quick start and expected outputs.

PART D: How to Build the Proper Docs

D1. Principles

1. Write for two readers: an engineer who must reproduce your work, and a judge who has 10 minutes. Put the answer first, and details below.
2. Every claim needs evidence. Replace adjectives ("fast", "accurate") with numbers and link to the log/table that proves them.
3. Docs-as-code. Keep everything in `docs` in the repo, in Markdown, versioned with the code. Diagrams as Mermaid, so they diff and never go stale.
4. One source of truth. Schemas live in code (Pydantic); docs are generated or copied from them, not retyped.
5. Trace it. Each gate → requirement → test → evidence (see Appendix A).
6. Be honest about failures. The guide *requires* analyzed failures. Judges reward clear diagnosis more than hidden problems.

D2. Document set and order of writing

Order	Document	When to write	Owner suggestion
1	Schemas (I/O, telemetry)	First, since they drive code and tests	Backend lead
2	SRD (Part C)	Before/during Phase 1–2	Tech lead
3	README + runbook	As soon as something runs; keep updating	Anyone
4	Architecture Brief (≤ 6 pages)	After Phase 3, finalize after Phase 5	Tech lead
5	Benchmark & Evaluation Report	Draft tables early; fill after runs	Eval lead
6	Telemetry & Observability Schema doc	Generated from the schema	Backend lead
7	Demo video	Last, from a clean run	Whole team

D3. Template: Architecture Brief (≤ 6 pages)

Page	Contents
------	----------

- 1 Problem, one-paragraph solution summary, the pipeline diagram, headline results (gate table with actuals)
- 2 Retrieval trigger logic: features, thresholds, decision order, budget; why these heuristics; timeline diagram of Example 1
- 3 Query decomposition + fusion: method, dedupe/merge rules, hybrid + RRF, corpus chunking and provenance
- 4 Session refinement + grounding: claim model, delta engine, verifier, uncertainty rules
- 5 Data provenance and telemetry: how a citation travels from chunk → claim → output; log schema summary; isolation guarantees
- 6 Trade-offs, failure modes, mitigations: what we rejected (e.g. agent frameworks) and why; known weaknesses; parsimony justification

Tips: one figure per page at most, no filler, and tables over paragraphs where possible.

D4. Template: Benchmarking & Evaluation Report

1. Setup: hardware, models, corpus size, test-set composition (counts per category), seeds.
2. Baseline definition: standard batch RAG on the *same* corpus and models (wait for full utterance → single query → answer). State it precisely so the comparison is fair.
3. Results table: baseline vs ours for G2–G5 plus latency (p50/p95 time to first token), tokens, cost.
4. Edge-case failure analyses (≥ 3). For each: *input stream* → *expected* → *observed* → *root cause* → *fix or open issue*. Good candidates: a premature trigger on a misleading first clause; a late constraint that contradicts an earlier claim; a topic missing from the corpus; a compound utterance with a shared context slot that got lost.
5. Ablations (≥ 2):
 - Hybrid vs dense-only retrieval (recall@k, latency);
 - Rule-based vs model-based controller (early-trigger rate, false triggers, latency, cost). Optional extras: with/without reranker; with/without verifier.
6. Threats to validity: small synthetic test set, judge subjectivity, overfitting risk.
7. Conclusion: which components earned their cost (ties back to architectural parsimony).

D5. Template: Telemetry & Observability Schema

- Event catalogue (table from SRD §C3.3): name, when emitted, fields, types, required/optional.
- Correlation IDs (, , ,).
- Example trace for Example 1 (full JSONL) and the derived timeline.
- How each gate is computed from logs (formulas): e.g. $G2 = \frac{\text{eligible queries where first } < \text{ , divided by all eligible queries.}}$
- Validation script and how to run it.

D6. Template: Demo video (≤ 5 minutes) [Proposed script]

Time	Segment	Show on screen
0:00 -0:3 0	The problem and the four capabilities	One slide, pipeline diagram
0:30 -1:3 0	Early retrieval: replay Example 1 in real time	Live transcript beside the telemetry timeline: → before utterance end
1:30 -2:3 0	Multi-intent decomposition: 3 sub-queries in parallel	Sub-query list, parallel retrieval bars, unified cited answer
2:30 -3:3 0	Late-detail refinement: Example 2, v1 → v2	Diff of claims, retained vs new citations, no full re-search
3:30 -4:0 0	Suppression: "repeat in two bullets"	event, zero searches
4:00 -4:3 0	Grounding + uncertainty: an out-of-corpus question	Explicit uncertainty; citation IDs traced back to the source
4:30 -5:0 0	Telemetry and reproducibility: show the one-command run and the metrics table	Terminal + gate results

Record from a clean run of the packaged container, so the demo doubles as proof of G1.

D7. README and runbook checklist

- What it is (2 lines) and the architecture diagram
- Quick start: prerequisites, the single command, expected output and where logs go
- Configuration table ()
- How to run the replay suite and compute metrics
- Repo map
- Troubleshooting (ports, model downloads, memory)
- Known limitations

D8. Documentation quality checklist

- Can a stranger run it in ≤ 5 minutes using only the README?
- Every number in the docs is reproducible from a log or script in the repo
- Every requirement in the SRD maps to a test
- Every gate has a measured result (or an honest gap plus a plan)
- Diagrams are Mermaid/text, so nothing is stale
- Architecture Brief is ≤ 6 pages; video $\leq 5:00$
- Terms used consistently (sub-intent, claim, answer version, provisional retrieval)
- No hardcoded test data or query-specific logic anywhere in the repo
- A teammate not on the feature has reviewed each doc

D9. Tooling suggestions

Markdown + Mermaid in the repo; (or your editor's export) for PDF of the Architecture Brief; /Excalidraw only if a diagram is too complex for Mermaid; a target () to build PDFs and regenerate the schema tables.

Appendix A: Traceability matrix (starter)

Gat	Requirements	Tests	Evidence artifact
e			
G1	FR-5.4, NFR-1	Clean-VM run	CI log, demo video
G2	FR-1.1–1.5	Controller replay suite	Benchmark report table, telemetry
G3	FR-2.1–2.4	Golden compound set	Benchmark report, decomposition logs
G4	FR-4.1, 4.4, 4.5, FR-3.3	Claim-audit script	Audit CSV, brief page 5
G5	FR-4.3, 4.6	Two-step refinement scenarios	logs, demo
G6	FR-5.1, 5.2, NFR-9	Log-chain validator	Telemetry schema doc

Appendix B: Suggested repository layout

```
streaming-live-rag/
├── README.md
├── docker-compose.yml  Dockerfile  .env.example  uv.lock
├── src/
│   ├── controller/    (features, policy, suppression)
│   ├── decomposer/
│   ├── retrieval/     (index, hybrid, rrf, rerank)
│   ├── synthesis/     (claims, refinement, verifier)
│   ├── session/       (ephemeral store)
│   ├── telemetry/     (events, emitter, metrics)
│   └── api/           (ws/sse, replay CLI)
├── tests/             (unit, replay scenarios, golden sets)
├── eval/              (datasets, scripts, ablation configs, results/)
├── docs/
│   ├── PRD.md  SRD.md  architecture_brief.md
│   └── benchmark_report.md  telemetry_schema.md  demo_script.md
```

Appendix C: Suggested phase plan (mirrors the guide's roadmap)

Phase	Build	Docs produced alongside
1. Framing & Foundation	Corpus audit, chunking, baseline hybrid retrieval, event schemas	Schemas, SRD v1, README skeleton
2. Controller & Stream Simulation	Chunk replay simulator, controller policy, timestamp logging	Controller section of brief, first G2 numbers
3. Multi-Intent & Fusion	Decomposer, parallel retrieval, RRF + dedupe	Decomposition/fusion section, G3 golden set
4. Refinement & State	Session store, delta engine, verifier, uncertainty	Refinement/grounding section, G4/G5 tests
5. Telemetry, Benchmark, Packaging	Full telemetry, held-out-style runs, ablations, Docker	Benchmark report, telemetry doc, demo video, final brief

Tip: freeze the schemas (Phase 1) and the test datasets (Phase 2) early. Nearly every later doc and metric depends on them.

Streaming Live RAG: Documentation Pack

Track: Theme 4, Streaming Live RAG (real-time incremental retrieval, multi-intent decomposition, state-preserving answer refinement)

Contents

- Part A: Detailed problem description (what the track really asks)
- Part B: Product Requirements Document (PRD)
- Part C: System Requirements / Specification Document (SRD)
- Part D: How to build the proper docs (templates, page budgets, review checklist)
- Appendix: Traceability matrix, repo layout, 2-week plan

Convention used below. Anything taken directly from the track guide is stated as fact. Anything marked [Proposed] is our own design choice or target, because the guide doesn't specify it.

Anything marked [Clarify] is an open question to confirm with the organizers.

PART A: Detailed Problem Description

A1. The problem in one paragraph

Normal RAG is turn-based: the user finishes, the system retrieves, the system answers. In voice and live-support settings, that means dead air. This track asks you to build an event-driven engine that consumes a transcript as it arrives (timestamped chunks such as 0.0 s, 0.8 s, 1.6 s, then "utterance end" at 2.1 s), starts retrieving before the user finishes, splits one rambling utterance into several sub-questions, updates its answer in place when the user adds a late detail, and never says anything that isn't backed by the supplied corpus, with full telemetry of everything it did.

A2. Why standard RAG fails here

Failure	What happens in a batch RAG	What the track wants instead
High latency	Waits for the whole utterance, then retrieves, so the user waits several seconds	Retrieval begins mid-utterance once intent is stable
Compound requests	One blended query such as "venue capacity cancellation catering" retrieves mush	Decompose into orthogonal sub-queries, retrieve in parallel
Late constraints	"Actually it was international" restarts the pipeline and discards context	Mutate only the affected claims; keep the rest and the citations

A3. The four required capabilities, explained

1. Listens incrementally (Retrieval Controller). For every incoming chunk, the controller makes one of three decisions:

- Wait: the intent is incomplete or unstable (the 0.0 s chunk "I need to plan a customer workshop in..." has no location yet).
- Retrieve: the intent is stable enough to search (at 0.8 s, "Pune for 30 people" gives a location and a capacity).
- No-Retrieval / Suppress: the request doesn't need the corpus at all (e.g. "repeat that in two bullets").

The hard part is the trade-off: retrieve too early and you burn compute on half-thoughts (the guide calls this *thrashing*); retrieve too late and you gain nothing over batch RAG.

2. Decomposes multi-intent queries. "...the cancellation policy and the catering options" contains two questions. Together with the earlier capacity question, that makes three sub-queries, dispatched in parallel. The trap is over-fragmenting: a simple question must not be split into near-duplicate queries (Pitfall #5).

3. Refines rather than restarts (Session-Only Refinement). The answer is a *versioned object*. When the user adds "the trip was international and the booking was made after travel", you do not rerun everything. You:

1. detect that this modifies an existing topic,
 2. query only for the delta (international travel rules, post-travel booking exceptions),
 3. rewrite only the affected claims,
 4. keep the earlier citations, add new ones, and bump Answer Version 1 → 2.
4. Guarantees corpus grounding. Every factual claim must carry a citation that exists in the corpus and actually supports the claim. If evidence is missing, the system must say so (an explicit uncertainty indicator) or ask a targeted clarification. Never invent a citation such as .

A4. The pipeline, stage by stage

Stream chunks → [1] Retrieval Controller → [2] Multi-Intent Decomposer
 → [3] Corpus Retrieval & Fusion → [4] Session-Aware Synthesis
 → Streamed answer + grounded citations + telemetry

Stage	Input	Output	Core challenge (from the guide)
1. Retrieval Controller	Transcript chunks + session state	Wait / Retrieve / No-Retrieve	Early latency gains vs premature noisy searches
2. Multi-Intent Decomposer	Utterance so far	List of search-ready sub-queries	Orthogonal questions without losing context or over-fragmenting
3. Retrieval & Fusion	Sub-queries	Ranked, deduplicated chunks per sub-query	Merge sources without diluting context or introducing contradictions
4. Session-Aware Synthesis	Evidence + prior answer version	Streamed answer, citations, uncertainty flags	Version tracking; mutate only affected claims
Cross-cutting: Telemetry	Everything	Structured logs	Clean telemetry under sub-second streaming

A5. Hard rules and what they mean for design

Rule (guide §3)	Design implication
Corpus isolation. No web, no third-party KBs, no parametric memory for facts.	An LLM may only <i>rephrase and combine retrieved text</i> . Prompt it with retrieved chunks only and verify outputs against them.
No hardcoding / precomputation. The replay benchmark is held-out and private.	No canned answers, no query-specific rules. Only generic indexing of the supplied corpus at startup is safe. [Clarify] whether indexing the corpus at build time is allowed.
Rigorous grounding. Every claim cites ; otherwise uncertainty or clarification.	Claim-level data model, plus a verifier that runs <i>before</i> anything is emitted.

Session-bound state. No cross-session profiling; memory is ephemeral. In-memory store keyed by `session_id`, wiped at session end; nothing persisted between runs.

Architectural parsimony. Extra agents and orchestration are judged on cost-to-performance. Prefer a single async pipeline. Every added component needs a measured latency/quality justification (ablations!).

A6. What the three worked examples teach

- Example 1 (incremental multi-intent): Timeline at 0.0 s Wait → 0.8 s provisional retrieve (`searches` , logged as `searches`) → 1.6 s decompose into 3 sub-queries and search in parallel → 2.1 s utterance end, synthesize one unified answer with citations. The output includes an `answer` field for anything not verifiable (e.g. catering policy for Venue A).
- Example 2 (late detail): Version 1 answers the base policy; the late detail triggers targeted delta queries; Version 2 keeps prior citations, adds new ones, and states the exception clearly.
- Example 3 (suppression): "Repeat your last answer in two bullets" → `repeat` ; zero vector searches, prior citations retained, nothing new fabricated.

A7. The six evaluation gates, decoded

Gate	Threshold	What judges will likely check	How we hit it
G1 Reproducibility	Pass/Fail	Fresh machine, one command, replay suite runs unattended	Docker Compose; pinned lockfiles; no manual steps; no online API needed at runtime (or clear key config)
G2 Early Retrieval	$\geq 80\%$ of eligible queries	timestamp < utterance-end timestamp, with a low false-trigger rate on no-retrieval cases	Feature-based controller; measure the false-trigger rate ourselves
G3 Multi-Intent	$\geq 70\%$ of compound utterances	≥ 2 distinct sub-intents identified and isolated	Decomposer with a golden set; guard against both under- and over-splitting
G4 Grounding	$\geq 85\%$ citation support; zero fabricated IDs	Sampled claims supported by cited chunks	Claim-level verifier; drop or convert unsupported claims to uncertainty
G5 Refinement	Verified state continuity	Late constraints update without clearing session or re-running full-corpus search	Claim-level answer versioning; delta retrieval only

G6 Telemetry	100% trace coverage	Timestamps, triggers, citations, version lineage, token cost for every request	Central event emitter; schema validated in tests
-----------------	------------------------	--	---

A8. Pitfalls → mitigations

Pitfall	Mitigation
Eager retrieval on noise	Stability check + per-utterance cap on provisional retrievals + reuse of cached results
Context loss on late constraints	Session store holds evidence and claims; delta-only queries
Citation hallucination	Whitelist of retrievable IDs; verifier rejects any ID not in the retrieved set
Ignoring presentation-only turns	Suppression classifier runs <i>first</i> in the controller
Over-fragmented sub-queries	Merge near-duplicates (similarity threshold); cap sub-query count

A9. Open questions to confirm with organizers

[Clarify]

1. Corpus format and size (files, scheme, how sections are delimited). Is the corpus given in advance or only at evaluation time?
2. Input format of the held-out replay: JSONL of ? Real-time pacing or accelerated replay?
3. Output contract: is the JSON in Example 1 the exact schema the harness reads? Where are telemetry logs read from (file, stdout, endpoint)?
4. LLMs: are external hosted LLM APIs allowed? Local models only? Any compute or cost cap? (This affects G1 heavily.)
5. Latency numbers: the guide gives none. Is there a target for time-to-first-token after utterance end?
6. G2 definition: what makes a query "eligible", and what false-trigger rate counts as "low"?
7. G4 method: human-judged or automated entailment? Granularity (sentence or claim)?
8. Baseline: is the "baseline pipeline" for the benchmarking report provided by organizers, or do we build one?
9. Clarification requests: does asking a clarifying question count as an acceptable alternative to an uncertainty flag under G4?

PART B: Product Requirements Document (PRD)

B1. Overview

Field	Value
Product	Streaming Live RAG Engine
Purpose	Deliver grounded, low-latency answers to live speech/typing by retrieving incrementally, decomposing compound requests, and refining answers in place
Primary consumer	Hackathon evaluation harness (automated replay), then judges reviewing the demo and docs
Secondary consumer (product vision)	Voice assistants and real-time support agents
Version	0.1 (draft)

B2. Goals and non-goals

Goals

1. Start retrieval before the utterance ends on $\geq 80\%$ of eligible queries, without thrashing on no-retrieval turns.
2. Correctly split compound utterances ($\geq 70\%$) into parallel sub-queries.
3. Produce answers where $\geq 85\%$ of factual claims are supported by valid citations, with zero fabricated IDs.
4. Refine answers in place when late constraints arrive, without clearing state or re-running full-corpus search.
5. Emit complete telemetry for every request (100% trace coverage).
6. Run from a single command on a clean machine.

Non-goals

- Speech-to-text (we consume *transcript chunks*; we don't build ASR). [Clarify]
- Cross-session memory, user profiling, personalization.
- Web search or any knowledge outside the supplied corpus.
- A polished end-user UI (a minimal demo UI or CLI is enough, mainly to show telemetry).
- Multi-agent orchestration beyond what ablations justify.

B3. Users and scenarios

Persona	Need	Example
Voice-assistant end user	Fast, correct answers while still talking; can change their mind mid-conversation	"Plan a workshop in Pune for 30... and the cancellation policy and catering?"
Support agent (assisted)	Grounded snippets with sources while a customer is speaking	Live policy lookup during a call
Evaluation harness	Deterministic replay, machine-readable outputs and logs	Automated gate scoring

Judge /
reviewer

Understand design, trade-offs,
evidence

Reads brief, report, watches demo

B4. User stories

ID	Story	Priori ty
US-1	As a speaker, I want the system to start searching once my intent is clear, so I don't wait after I finish.	Must
US-2	As a speaker, I want each part of my compound request answered, so I don't have to repeat myself.	Must
US-3	As a speaker, I want my earlier answer updated when I add a detail, so the conversation flows naturally.	Must
US-4	As a reader, I want every fact linked to a source, and gaps flagged, so I can trust the answer.	Must
US-5	As a speaker, I want formatting requests (shorten, bullets, translate) handled without a new search.	Must
US-6	As an evaluator, I want structured logs of all decisions, so I can score the run.	Must
US-7	As an evaluator, I want to run everything with one command.	Must
US-8	As a developer, I want to compare controller and retrieval variants (ablations).	Shoul d
US-9	As a demo viewer, I want to see the timeline, version history, and citations live.	Shoul d
US-10	As a speaker, I want a clarifying question when evidence is ambiguous.	Coul d

B5. Functional requirements

Retrieval Controller

ID	Requirement	Priori ty
FR-1.1	Process timestamped transcript chunks in arrival order, emitting a decision (,) per chunk with a reason code.	Must
FR-1.2	Trigger a provisional retrieval when the intent is stable (topic plus at least one concrete constraint).	Must

FR-1.3	Suppress retrieval for presentation-only requests (repeat, shorten, reformat, translate, summarize prior output) and log .	Must
FR-1.4	Limit provisional retrievals per utterance and reuse results if the final query is equivalent.	Should
FR-1.5	Reconcile provisional results with the final utterance at utterance end (keep, extend, or discard).	Must

Multi-Intent Decomposition

ID	Requirement	Priority
FR-2.1	Extract independent sub-queries from a compound utterance while preserving shared context (e.g. "Pune", "30 people").	Must
FR-2.2	Dispatch sub-query retrievals concurrently.	Must
FR-2.3	Merge near-duplicate sub-queries; never fragment a single-intent utterance.	Must
FR-2.4	Track each sub-intent as an object (, text, status) for coverage checks.	Must

Retrieval and Fusion

ID	Requirement	Priority
FR-3.1	Retrieve only from the supplied corpus using hybrid (sparse + dense) scoring.	Must
FR-3.2	Fuse ranked lists (e.g. RRF), deduplicate overlapping chunks, and rerank.	Must
FR-3.3	Preserve and provenance on every chunk.	Must

Synthesis, Refinement, Grounding

ID	Requirement	Priority
FR-4.1	Generate answers as claims, each with ≥ 1 citation .	Must
FR-4.2	Stream the answer incrementally and support versioning ().	Must
FR-4.3	On a late constraint, identify affected claims, run delta retrieval only, mutate those claims, retain the rest, increment the version.	Must

FR-4.4	Verify each claim against its cited chunk before emitting; drop or convert failures to an uncertainty statement.	Must
FR-4.5	Every sub-intent ends with either a supported claim or an explicit uncertainty entry.	Must
FR-4.6	Session memory is ephemeral, scoped to , and cleared at session end.	Must
FR-4.7	Optionally emit a clarifying question when evidence is insufficient or ambiguous.	Could

Observability and Packaging

ID	Requirement	Priority
FR-5.1	Emit structured telemetry events for all decisions (see SRD §3.3).	Must
FR-5.2	Record per-request latency breakdown and token/cost estimates.	Must
FR-5.3	Provide a replay runner to execute test streams and produce metrics.	Must
FR-5.4	Launch with (or a single CLI).	Must
FR-5.5	Provide a minimal live view (CLI or web) showing timeline, versions, citations.	Should

B6. Success metrics

Metric	Target	Source
Early-retrieval rate	$\geq 80\%$ of eligible queries	G2
False-trigger rate on no-retrieval cases	[Proposed] $\leq 10\%$ (confirm)	G2
Multi-intent identification	$\geq 70\%$ of compound utterances	G3
Citation support	$\geq 85\%$ of sampled claims	G4
Fabricated citation IDs	0	G4
State continuity on refinements	100% of refinements avoid session reset and full re-search	G5
Trace coverage	100%	G6
Clean-machine one-command replay	Pass	G1

[Proposed] Time from utterance end to first answer token ≤ 1.0 s at p50 (internal target)

Team

B7. Scope: MVP vs stretch

MVP (must be fully working)

- Rule-based controller with stability heuristics and suppression
- LLM- or rule-assisted decomposer
- Hybrid retrieval + RRF + dedupe
- Claim-level synthesis, verifier, uncertainty
- Answer versioning and delta refinement
- Telemetry JSONL, replay runner, Docker packaging

Stretch

- Model-based controller (for the ablation)
- Cross-encoder reranker
- Live dashboard, clarification questions
- Speculative pre-generation of answer drafts

B8. Assumptions, dependencies, risks

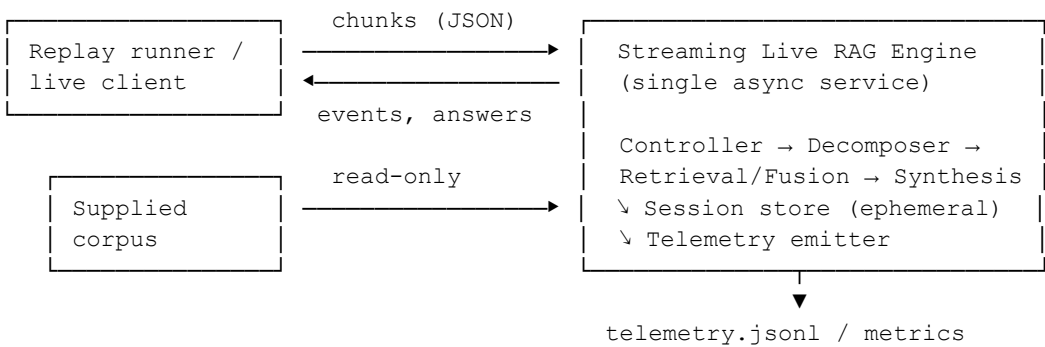
Type	Item	Mitigation
Assumption	Input is text chunks with timestamps	Confirm with organizers
Assumption	Corpus fits in memory and can be indexed at startup	Lightweight indexes, persisted only within the container run
Dependency	LLM for synthesis/decomposition	Pluggable interface; local fallback; determinism (temp 0)
Risk	Hidden replay data differs from our tests	Build diverse synthetic test streams; avoid overfitting/hardcoding
Risk	Verifier too strict, dropping good claims	Tune thresholds on our held-out set; report precision/recall
Risk	Latency from LLM calls	Cap calls per utterance; run decomposition and retrieval concurrently
Risk	Over-engineering (parsimony rule)	Ablations justify each component; drop what doesn't pay off

B9. Deliverables (from the guide's §8)

1. Reproducible repository (source, pinned lockfiles, env templates, one-command run)
 2. System Architecture Brief (≤ 6 pages)
 3. Benchmarking & Evaluation Report (vs baseline; ≥ 3 analyzed edge-case failures; 2 ablations)
 4. System Demonstration Video (≤ 5 minutes)
 5. Telemetry & Observability Schema
-

PART C: System Requirements / Specification Document (SRD)

C1. System context



External interfaces: input event stream; output event stream; corpus (read-only); telemetry sink; optional LLM endpoint ([Clarify] whether external APIs are allowed).

C2. Architecture and recommended stack

[Proposed]

Layer	Recommendation	Why
Runtime	Python 3.11,	Concurrency for parallel sub-query retrieval; one process keeps overhead low
API	FastAPI with WebSocket/SSE for live; CLI for replay	Streaming in and out; harness-friendly
Sparse index	BM25 (, SQLite FTS5, or Tantivy)	Exact keyword and ID matches
Dense index	Small local embedding model (e.g. bge-small/e5-small) + FAISS/hnswlib	Offline, reproducible, fast
Fusion	Reciprocal Rank Fusion (k≈60)	Simple, robust, no training
Reranker	Optional cross-encoder (stretch; ablation)	Precision gain vs latency cost
LLM	Pluggable interface (local or hosted), temperature 0, JSON-mode outputs	Decomposition and synthesis; swap for cost/latency tests
State	In-memory dict keyed by	Satisfies ephemeral-memory rule

Schema Pydantic models Validation for I/O and telemetry
s

Packagi Dockerfile + , lockfile G1
ng (/ with hashes)

Testing , replay harness, metrics scripts Gates G2–G6

Parsimony stance: start with a *single pipeline, no agent framework*. Add anything else only with an ablation that shows it earns its latency and cost.

C3. Interface specifications

C3.1 Input event [Proposed; align to organizer format]

```
{
  "session_id": "s-001",
  "seq": 3,
  "timestamp_s": 1.6,
  "text": "...the cancellation policy and the catering options.",
  "type": "chunk"           // "chunk" | "utterance_end" | "session_end"
}
```

C3.2 Output event (answer record). Follows the guide's Example 1 and extends it

```
{
  "session_id": "s-001",
  "answer_version": 1,
  "retrieval_events": [
    { "timestamp_s": 0.8, "query": "Pune workshop venue capacity 30", "trigger":
"provisional" },
    { "timestamp_s": 1.6, "query": "cancellation policy workshop venues Pune",
"trigger": "multi_intent" }
  ],
  "sub_queries": ["venue capacity for 30 attendees in Pune", "cancellation terms
and refund policies", "on-site and external catering options"],
  "answer": "For a 30-person workshop in Pune, documented options include ...",
  "claims": [
    { "claim_id": "c1", "sub_intent_id": "si1", "text": "...", "citations": ["Doc_12
$2"], "status": "supported" }
  ],
  "citations": ["Doc_12 $2", "Doc_31 $4", "Doc_09 $1"],
  "uncertainty": "Catering accommodation policies for Venue A could not be verified
from the retrieved corpus."
}
```

C3.3 Telemetry event schema (one JSON object per line)

```
{
  "ts_wall": "2026-09-21T10:15:02.123Z",    // absolute time
  "ts_stream_s": 0.8,                       // position in the replayed stream
  "session_id": "s-001",
  "request_id": "r-001",
  "event": "retrieval_started",
  "payload": { },
  "latency_ms": null,
```

```
"tokens_in": 0, "tokens_out": 0, "cost_usd": 0.0
}
```

Required event types

Event	Payload highlights
	seq, text, ts_stream_s
	decision (WAIT/RETRIEVE/NO_RETRIEVAL), reason, features (stability score, drift, slot list)
	sub_intents, merged_duplicates_count, latency_ms
/	sub_intent_id, query, trigger (provisional/multi_intent/final/delta), top-k chunk IDs + scores, latency_ms
	method (RRF), input lists, dedupe_removed
/	answer_version, first-token latency, total tokens
	claims_checked, supported, dropped, invalid_citation_count
	from_version, to_version, affected_claim_ids, delta_queries, retained_claim_ids
	reason (e.g.)
	sub_intent_id, cause
	memory_cleared: true

Trace-coverage rule (G6): every request has a complete chain from to , with no gaps, plus version lineage and cost fields.

C4. Component specifications

C4.1 Retrieval Controller

Inputs: chunk, accumulated utterance text, session state. Outputs: decision + reason + features.

Decision order (evaluate top to bottom, first match wins):

1. Suppression check. Presentation-only intent (verbs like repeat, shorten, rephrase, translate, "in N bullets") that refers to a prior answer → ().
2. Refinement check. Utterance refers to an existing topic and adds a constraint ("actually", "also", "the trip was...") → in mode.
3. Stability check. Retrieve provisionally when all hold [Proposed heuristics, tune on our data]:
 - a topic/intent head is present (e.g. "workshop", "reimbursement");
 - ≥ 1 concrete constraint slot filled (location, count, date, product...);
 - the chunk does not end in an unfinished construction (dangling "and I", "in...", "for the");

- the intent hypothesis has drifted less than a threshold from the last provisional query (embedding or token overlap).

4. Otherwise → .

Budgets: max N provisional retrievals per utterance (start with 2–3); skip re-retrieval if the new query is $\geq \tau$ similar to a cached one. Reconciliation at utterance end: compare final sub-intents with provisional ones; reuse the cache when equivalent, run extra retrievals only for uncovered sub-intents, and drop provisional results that no longer match.

Variants for ablation: rule-based (default) vs. small-model-based classifier.

C4.2 Multi-Intent Decomposer

- Input: utterance text (possibly partial) + session context. Output: list of
- Method: LLM with strict JSON schema, few-shot examples; deterministic fallback (split on coordinating conjunctions and enumerations, then filter by noun-phrase content).
- Rules: inherit shared context slots into each sub-query (city, headcount); merge sub-queries whose embedding similarity $> \tau$; cap at K sub-queries [Proposed K = 5]; single-intent utterances yield exactly one sub-query.

C4.3 Retrieval and Fusion

1. Query each sub-intent concurrently against sparse and dense indexes (top-k each, e.g. k = 20).
2. Fuse with RRF:
3. Deduplicate near-identical or overlapping chunks (same or high text overlap).
4. Optional rerank (cross-encoder) → keep top-m per sub-intent (e.g. m = 3–5).
5. Return chunks with , , text, scores.

Corpus ingestion: parse into chunks aligned to sections, so every chunk has a stable . Build both indexes at startup or at container build [Clarify which is permitted].

C4.4 Session-Aware Synthesis and Refinement

Session state (ephemeral):

```
Session {
  session_id, answer_version,
  sub_intents: {id → SubIntent},
  evidence: {sub_intent_id → [chunk_ids]},
  claims: [Claim{id, text, sub_intent_id, citations[], status, introduced_in_v,
updated_in_v}],
  answer_history: [version snapshots (diffs)]
}
```

Synthesis: prompt the LLM with *only* retrieved chunks for each sub-intent; require output as claims with citation IDs chosen from the provided set; temperature 0.

Refinement (delta engine):

1. Extract the new constraint(s) from the latest utterance.
2. Map to affected sub-intents/claims (topic match + slot overlap).
3. Run delta retrieval only for the new constraint (e.g.).
4. Regenerate or edit only affected claims; add new ones; mark contradicted claims (kept in history).

5. Retain unaffected claims and citations; increment ; log

Grounding verifier (runs before emit):

1. ID validity: each citation $\in$ the set of retrieved chunk IDs $\in$ corpus index. Otherwise reject.
2. Support check: claim text is entailed by the cited chunk (embedding/lexical overlap gate, plus an NLI or LLM-judge check for uncertain cases).
3. On failure: rewrite once using the evidence, else drop the claim and record for that sub-intent.
4. Coverage invariant: every sub-intent has ≥ 1 supported claim or an explicit uncertainty entry.

Uncertainty triggers: top retrieval score below threshold, no chunk covering the sub-intent, verifier failure, or conflicting sources.

C4.5 Telemetry Emitter

- Async, non-blocking queue writing JSONL; never awaited on the critical path.
- Every component emits through one helper, so no event is forgotten.
- Token and cost accounting wraps the LLM client.
- A summary script converts logs into a metrics table (G2–G6).

C5. Non-functional requirements

ID	Requirement	Target
NFR-1	Reproducibility	Single command; pinned deps; deterministic seeds; temp 0
NFR-2	Time to first answer token after utterance end	[Proposed] $p50 \leq 1.0$ s, $p95 \leq 2.0$ s
NFR-3	Controller decision time per chunk	[Proposed] ≤ 50 ms (rule-based)
NFR-4	Retrieval latency per sub-query	[Proposed] ≤ 200 ms on the provided corpus
NFR-5	Isolation	No outbound calls except the (permitted) LLM endpoint; no web; corpus read-only
NFR-6	Privacy	No persistence across sessions; memory cleared on
NFR-7	Robustness	Handles out-of-order/duplicate chunks, empty chunks, very long utterances, corpus-missing topics
NFR-8	Cost	Track tokens per request; report cost-to-quality in the benchmark
NFR-9	Observability	100% trace coverage; schema-validated logs

NFR -10	Maintainability	Type hints, schema validation, ≥ 1 test per requirement group
---------	-----------------	---

C6. Verification matrix

Requirement group	Test type	Pass criterion
Controller (FR-1.x)	Replay streams with labeled ideal trigger points	≥ 80% early trigger; false-trigger rate within target
Decomposition (FR-2.x)	Golden compound utterances	≥ 70% correct; zero over-fragmentation on single-intent set
Retrieval (FR-3.x)	Golden question→chunk pairs	Recall@k and MRR; hybrid ≥ dense-only (ablation)
Grounding (FR-4.1, 4.4, 4.5)	Sampled-claim audit (script + manual)	≥ 85% supported; 0 invalid IDs
Refinement (FR-4.3)	Two-step scenarios	Version increments; no full-corpus re-search; unaffected claims unchanged
Suppression (FR-1.3)	Presentation-only prompts	0 retrievals; citations preserved
Telemetry (FR-5.x)	Log validator	100% chain completeness
Packaging (FR-5.4)	Clean VM / fresh container	One command reproduces results

C7. Deployment

- `builds` builds the image, mounts the corpus read-only, starts the service and (optionally) runs the replay suite, writing `logs` and `metrics` to a mounted `data` folder.
- `docs` documents all configuration (model names, thresholds, LLM endpoint).
- README gives a 5-minute quick start and expected outputs.

PART D: How to Build the Proper Docs

D1. Principles

1. Write for two readers: an engineer who must reproduce your work, and a judge who has 10 minutes. Put the answer first, and details below.

2. Every claim needs evidence. Replace adjectives ("fast", "accurate") with numbers and link to the log/table that proves them.
3. Docs-as-code. Keep everything in the repo, in Markdown, versioned with the code. Diagrams as Mermaid, so they diff and never go stale.
4. One source of truth. Schemas live in code (Pydantic); docs are generated or copied from them, not retyped.
5. Trace it. Each gate → requirement → test → evidence (see Appendix A).
6. Be honest about failures. The guide *requires* analyzed failures. Judges reward clear diagnosis more than hidden problems.

D2. Document set and order of writing

Order	Document	When to write	Owner suggestion
1	Schemas (I/O, telemetry)	First, since they drive code and tests	Backend lead
2	SRD (Part C)	Before/during Phase 1–2	Tech lead
3	README + runbook	As soon as something runs; keep updating	Anyone
4	Architecture Brief (≤ 6 pages)	After Phase 3, finalize after Phase 5	Tech lead
5	Benchmark & Evaluation Report	Draft tables early; fill after runs	Eval lead
6	Telemetry & Observability Schema doc	Generated from the schema	Backend lead
7	Demo video	Last, from a clean run	Whole team

D3. Template: Architecture Brief (≤ 6 pages)

Page	Contents
1	Problem, one-paragraph solution summary, the pipeline diagram, headline results (gate table with actuals)
2	Retrieval trigger logic: features, thresholds, decision order, budget; why these heuristics; timeline diagram of Example 1
3	Query decomposition + fusion: method, dedupe/merge rules, hybrid + RRF, corpus chunking and provenance
4	Session refinement + grounding: claim model, delta engine, verifier, uncertainty rules
5	Data provenance and telemetry: how a citation travels from chunk → claim → output; log schema summary; isolation guarantees

- Trade-offs, failure modes, mitigations: what we rejected (e.g. agent frameworks) and why; known weaknesses; parsimony justification

Tips: one figure per page at most, no filler, and tables over paragraphs where possible.

D4. Template: Benchmarking & Evaluation Report

- Setup: hardware, models, corpus size, test-set composition (counts per category), seeds.
- Baseline definition: standard batch RAG on the *same* corpus and models (wait for full utterance → single query → answer). State it precisely so the comparison is fair.
- Results table: baseline vs ours for G2–G5 plus latency (p50/p95 time to first token), tokens, cost.
- Edge-case failure analyses (≥ 3). For each: *input stream* → *expected* → *observed* → *root cause* → *fix or open issue*. Good candidates: a premature trigger on a misleading first clause; a late constraint that contradicts an earlier claim; a topic missing from the corpus; a compound utterance with a shared context slot that got lost.
- Ablations (≥ 2):
 - Hybrid vs dense-only retrieval (recall@k, latency);
 - Rule-based vs model-based controller (early-trigger rate, false triggers, latency, cost). Optional extras: with/without reranker; with/without verifier.
- Threats to validity: small synthetic test set, judge subjectivity, overfitting risk.
- Conclusion: which components earned their cost (ties back to architectural parsimony).

D5. Template: Telemetry & Observability Schema

- Event catalogue (table from SRD §C3.3): name, when emitted, fields, types, required/optional.
- Correlation IDs (, , ,).
- Example trace for Example 1 (full JSONL) and the derived timeline.
- How each gate is computed from logs (formulas): e.g. $G2 = \frac{\text{eligible queries where first } < \text{ , divided by all eligible queries.}}$
- Validation script and how to run it.

D6. Template: Demo video (≤ 5 minutes) [Proposed script]

Time	Segment	Show on screen
0:00 –0:30 0	The problem and the four capabilities	One slide, pipeline diagram
0:30 –1:30 0	Early retrieval: replay Example 1 in real time	Live transcript beside the telemetry timeline: → before utterance end
1:30 –2:30 0	Multi-intent decomposition: 3 sub-queries in parallel	Sub-query list, parallel retrieval bars, unified cited answer

2:30 -3:3 0	Late-detail refinement: Example 2, v1 → v2	Diff of claims, retained vs new citations, no full re-search
3:30 -4:0 0	Suppression: "repeat in two bullets"	event, zero searches
4:00 -4:3 0	Grounding + uncertainty: an out-of-corpus question	Explicit uncertainty; citation IDs traced back to the source
4:30 -5:0 0	Telemetry and reproducibility: show the one-command run and the metrics table	Terminal + gate results

Record from a clean run of the packaged container, so the demo doubles as proof of G1.

D7. README and runbook checklist

- What it is (2 lines) and the architecture diagram
- Quick start: prerequisites, the single command, expected output and where logs go
- Configuration table ()
- How to run the replay suite and compute metrics
- Repo map
- Troubleshooting (ports, model downloads, memory)
- Known limitations

D8. Documentation quality checklist

- Can a stranger run it in ≤ 5 minutes using only the README?
- Every number in the docs is reproducible from a log or script in the repo
- Every requirement in the SRD maps to a test
- Every gate has a measured result (or an honest gap plus a plan)
- Diagrams are Mermaid/text, so nothing is stale
- Architecture Brief is ≤ 6 pages; video $\leq 5:00$
- Terms used consistently (sub-intent, claim, answer version, provisional retrieval)
- No hardcoded test data or query-specific logic anywhere in the repo
- A teammate not on the feature has reviewed each doc

D9. Tooling suggestions

Markdown + Mermaid in the repo; (or your editor's export) for PDF of the Architecture Brief; /Excalidraw only if a diagram is too complex for Mermaid; a target () to build PDFs and regenerate the schema tables.

Appendix A: Traceability matrix (starter)

Gat e	Requirements	Tests	Evidence artifact
G1	FR-5.4, NFR-1	Clean-VM run	CI log, demo video
G2	FR-1.1–1.5	Controller replay suite	Benchmark report table, telemetry
G3	FR-2.1–2.4	Golden compound set	Benchmark report, decomposition logs
G4	FR-4.1, 4.4, 4.5, FR-3.3	Claim-audit script	Audit CSV, brief page 5
G5	FR-4.3, 4.6	Two-step refinement scenarios	logs, demo
G6	FR-5.1, 5.2, NFR-9	Log-chain validator	Telemetry schema doc

Appendix B: Suggested repository layout

```
streaming-live-rag/
├── README.md
├── docker-compose.yml  Dockerfile  .env.example  uv.lock
├── src/
│   ├── controller/    (features, policy, suppression)
│   ├── decomposer/
│   ├── retrieval/     (index, hybrid, rrf, rerank)
│   ├── synthesis/     (claims, refinement, verifier)
│   ├── session/       (ephemeral store)
│   ├── telemetry/     (events, emitter, metrics)
│   └── api/           (ws/sse, replay CLI)
├── tests/             (unit, replay scenarios, golden sets)
├── eval/              (datasets, scripts, ablation configs, results/)
├── docs/
│   ├── PRD.md  SRD.md  architecture_brief.md
│   └── benchmark_report.md  telemetry_schema.md  demo_script.md
```

Appendix C: Suggested phase plan (mirrors the guide's roadmap)

Phase	Build	Docs produced alongside
-------	-------	-------------------------

1. Framing & Foundation	Corpus audit, chunking, baseline hybrid retrieval, event schemas	Schemas, SRD v1, README skeleton
2. Controller & Stream Simulation	Chunk replay simulator, controller policy, timestamp logging	Controller section of brief, first G2 numbers
3. Multi-Intent & Fusion	Decomposer, parallel retrieval, RRF + dedupe	Decomposition/fusion section, G3 golden set
4. Refinement & State	Session store, delta engine, verifier, uncertainty	Refinement/grounding section, G4/G5 tests
5. Telemetry, Benchmark, Packaging	Full telemetry, held-out-style runs, ablations, Docker	Benchmark report, telemetry doc, demo video, final brief

Tip: freeze the schemas (Phase 1) and the test datasets (Phase 2) early. Nearly every later doc and metric depends on them.